ウォレットアドレスの匿名性を維持した PKIに基づく暗号資産の保有証明

石井 龍[1,a)]　野村 健太[1]　佐古 健太郎[1]　林田 淳一郎[1]　永井 達也[1]　齋藤 恆和[1]　渡邉 卓弥[1]
高田 雄太[1]　熊谷 裕志[1]　神薗 雅紀[1,2]

概要：金融サービスでは，KYC 等により本人確認されたユーザについて，オンチェーン上の暗号資産の保有状況を審査材料として確認したい場面がある．このためには，Web2 PKI 上で識別される実世界のユーザと，ウォレットアドレスに紐づくオンチェーン情報を安全に結び付ける必要がある．しかし，ウォレットアドレスを実世界のユーザと対応付けて検証者へ提示すると，公開台帳上の残高や取引履歴から過去および将来のオンチェーン活動を追跡可能になるというプライバシ上の課題が生じる．本研究では，ユーザ，発行者，検証者の三者からなるモデルに基づき，発行者がオンチェーン情報を取得して資産に関するポリシー評価を行い，その評価結果を主とするクレデンシャルを発行する PKI に基づく保有証明方式を提案する．提案方式では，ユーザは検証者にアドレス情報を直接与えずに暗号資産の保有条件の充足結果を属性として提示することができる．さらに，クレデンシャル形式として JWT，SD-JWT，BBS 署名の 3 つについてそれぞれ認証シーケンスを設計し実装評価した結果，SD-JWT 方式は既存 Web 技術との親和性を維持すること，BBS 方式は計算コストの増加を伴うものの，holder binding を課さない場合に複数提示間の非リンク性を提供できることが明らかになった．

PKI-based Proof of Cryptocurrency Possession Preserving Wallet Address Privacy

Ryu Ishii[1,a)]　Kenta Nomura[1]　Kentaro Sako[1]　Junichiro Hayata[1]　Tatsuya Nagai[1]
Tsunekazu Saito[1]　Takuya Watanabe[1]　Yuta Takata[1]　Hiroshi Kumagai[1]　Masaki Kamizono[1,2]

Abstract: In financial services, there are situations in which the on-chain cryptocurrency holdings of users whose identities have been verified through KYC procedures are considered as part of a screening process. This requires securely linking a real-world user identified through a Web2 PKI with on-chain information associated with the user's wallet address. However, if a wallet address is linked to a real-world user and disclosed to a verifier, the verifier may be able to track the user's past and future on-chain activities, including balances and transaction histories recorded on the public ledger. In this paper, we propose a PKI-based proof-of-possession scheme based on a three-party model consisting of a user, an issuer, and a verifier. In the proposed scheme, the issuer retrieves on-chain information, evaluates asset-related policies, and issues a credential primarily containing the evaluation result. The user can present, as an attribute, whether a cryptocurrency holding condition is satisfied without directly disclosing the wallet address to the verifier. We further design authentication sequences using three credential formats—JWT, SD-JWT, and BBS signatures—and evaluate their implementations. The results show that the SD-JWT-based scheme maintains compatibility with existing Web technologies, whereas the BBS-based scheme without holder binding provides unlinkability across multiple presentations at the cost of increased computational overhead.

[1] デロイト トーマツ サイバー合同会社,
Deloitte Tohmatsu Cyber LLC
[2] 一般社団法人デロイト トーマツ戦略研究所,
Deloitte Tohmatsu Strategy Institute
[a)] ryu.ishii@tohmatsu.co.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

金融関連サービスでは，不正利用防止の観点から，本人確認（KYC: Know Your Customer）やユーザ属性の確認が求められる [1], [2]．また，与信審査では収入や預金，有価証券等の資産情報が判断材料として用いられてきた．暗号資産の普及に伴い，その保有状況を金融サービスの審査材料として利用することも考えられ，暗号資産の保有状況を第三者が確認できる仕組みが必要となる．本研究では，複数の金融機関に対して並行して申し込まれる予備的な審査（仮審査）を想定するが，条件を満たすか否かのみが判断されれば足り，かつ後続の本審査に進むのは通常一社であるため，申込先すべてにウォレットアドレスを開示することは過剰な情報提供となる．

ウォレットアドレスおよび当該アドレスに紐づく残高や取引履歴は公開ブロックチェーン上で参照可能であるため，ユーザがウォレットアドレスを実世界の本人情報とともに検証者へ提示すると，検証者は当該ユーザとオンチェーン活動との対応関係を把握し，過去および将来の資産状況や取引履歴を追跡できる可能性がある．特に金融用途においては，サービス利用に必要な条件のみを確認したい場合であっても，アドレスの開示により審査に不要な資産情報や取引履歴まで推測され得る．また，同一ユーザの複数回提示によるリンク可能性も課題である．例えば，ブロックチェーン上の活動はトランザクション解析によりリンクされ得ることが知られており [3]，認証プロトコルにおいても提示情報の構造によりリンク可能性が生じる．

ユーザ情報を適切に認証へ利用するため，近年では Verifiable Credential（VC）[4] や Selective Disclosure JWT（SD-JWT）[5] といった，ユーザは必要な属性のみを選択的開示できる認証技術が提案されている．

これらの方式を Web3 のウォレット情報と組み合わせて実用的なサービスとして構築する場合，いくつかの問題が残されている．第一に，オンチェーン情報の取得とその正当性の検証をどの主体が担うべきかという問題がある．第二に，金融規制や KYC 要件との整合性を考慮した上で，ユーザのプライバシ保護と追跡可能性をどの程度両立するかという設計上のトレードオフが存在する．第三に，発行者がどの程度ユーザの責任を引き受けるか，および検証者がどの程度検証コストを負担するかといった，システム全体の責任分担の設計が必要である．さらに，既存研究では上記の方式を同一モデルで比較した例は少なく，選択的開示や非リンク性といったプライバシ保護機能と，実装コストや処理性能とのトレードオフも無視できない．

本研究では，ユーザ，発行者，検証者の三者からなるモデルに基づき，ウォレット情報に基づく属性を Web2 PKI 上で提示・検証する方式を提案する．本モデルでは，図 1 に示すように発行者は，あらかじめ定義した資産保有条件に関する述語集合をオンチェーン情報に対して評価し，その評価結果をクレデンシャルとして発行する．検証者は，サービスに必要な述語の評価結果のみをユーザに要求し，自身の判定に利用する．したがって，本方式は規制要件そのものを新たに定義するものではなく，発行者による評価結果をプライバシを保護しつつ検証者へ伝達する仕組みを提供する．

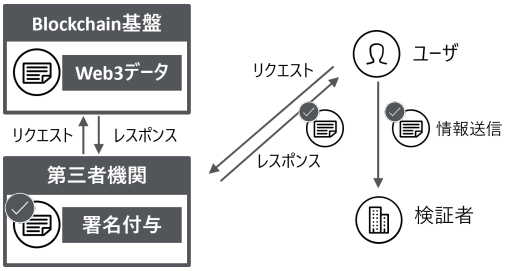


図 1 本研究のモデル．本研究では第三者機関を信頼の起点とする．

さらに本研究では，認証情報の実現方式として，(1) JWT を用いたベースライン方式，(2) SD-JWT を用いた選択的開示方式，(3) BBS 署名を用いた非リンク提示方式，の三方式を対象とし，それぞれの安全性および実装特性を比較する．これにより，Web2–Web3 連携認証における設計上のトレードオフを明らかにすることを目的とする．

本論文の主な貢献は以下の通りである．

- Web2 鍵および Web3 ウォレット情報を統合した三者モデルを定義し，オンチェーン情報の取得と検証を含む認証プロトコルを整理した．
- JWT，SD-JWT，BBS の三方式に基づく認証シーケンスを設計し，それぞれの安全性要件および脅威モデルとの対応関係を明確化した．
- 同一モデル下での実装評価により，BBS は JWT/SD-JWT と比較して処理コストが 2 桁大きく，特に提示生成および検証では 100 ms 程度要し，holder binding を課さない場合に限り送信データサイズの削減と非リンク提示が可能であることを示した．

2. 関連研究

本節では本研究の関連研究として，選択的開示方式，クレデンシャルの非譲渡性についてそれぞれ説明する．

2.1 選択的開示方式

JWT の標準仕様では属性の部分開示は想定されておらず，検証者に対して不要な属性も含めて提示されるため，選択的開示可能な提示方式について説明する．

SD-JWT. SD-JWT（Selective Disclosure for JWT） [5] とは，各クレームのハッシュ値を署名対象に含めることで，提示時に一部の属性のみを開示可能とする方式である．

JSON-LD+BBS. JSON-LD VC と BBS 署名を組合せ

© 2026 Information Processing Society of Japan

る方式 [6], [7] では，署名のランダマイズおよび知識証明を通じて，署名付きメッセージ集合の部分開示および非リンク性を実現する．

2.2 クレデンシャルの非譲渡性

クレデンシャルの非譲渡性とは，正当な保有者以外が有効な提示を生成できない性質であり，holder binding はその代表的な実現手法である．既存研究を実現方法の観点から便宜上，(1) クレデンシャル内在型，(2) 検証過程のゼロ知識証明型，(3) 外部デバイス統合型の 3 種類に分類する．

表 1 に既存研究と本研究の方式を定性的に整理する．既存研究は非譲渡性と非リンク性を両立するが，既存の Web2 証明書をそのまま利用することは想定されていない．一方本研究は，既存の Web2 証明書を holder binding に用いる key binding (KB) 構成を対象とする．この構成では，提示時に同一の証明書を検証者へ送付する必要があり，非譲渡性と非リンク性を同時には満たせない[*1]．

一方，検証者が監査可能性の高い既存の PKI 基盤で非譲渡性を要求できる運用，検証者への本人確認が必須でない仮審査の段階では非リンク性を優先する運用のどちらも選択可能である．

(1) クレデンシャル内在型

Lehmann ら [9] および Yamamoto ら [10] は，保有者秘密に依存する値をクレデンシャルの署名対象へ組み込む方式を提案している．保有者の秘密値に依存する情報を BBS 署名の対象に含めることで，保有者に結び付いたクレデンシャルを実現している．いずれも提示時には，保有者秘密の知識証明を必要とするため，第三者は有効な提示を生成できない．この方式は非譲渡性を暗号的に保証しやすく，ゼロ知識証明を併用することで非リンク性も維持しやすい．

(2) 検証過程のゼロ知識証明型

Paquin ら [8] は，既存の JWT や mDL を変更せず，その署名検証過程自体をゼロ知識証明として提示する方式を提案している．この方式では，属性の選択的開示と holder binding を ZK 回路内で同時に扱えるため，既存の発行基盤を保ったまま導入できる．一方で，クレデンシャルごとに一度だけ必要となる事前計算のコストが大きく，また trusted setup を要する点が課題である．

(3) 外部デバイス統合型

Hesse ら [11] は，物理カードと他人のデジタルクレデンシャルを組合せる mix-and-match 攻撃の防止を主目的とし，生体情報でカードに認証し，クレデンシャルを人間に束縛する方式を提案している．

[*1] これは本研究の証明書提示型 key binding による制約であり，[8] のように保有者鍵の所持を公開鍵秘匿しつつゼロ知識証明することで固定的な識別情報によるリンク可能性を回避できる．ただし，5 節の環境で https://github.com/microsoft/crescent-credentials ([8] 公開実装) にある保有者鍵の所持証明 (ECDSA-P256, mDL 形式) を実行した結果，クレデンシャルごとに 40.3 s の事前計算と回路ごとに 675 MB のパラメータ生成を要するため，仮審査のような一時的な手続きには適合しにくい．

表 1 既存研究と本研究の方式の比較．

方式	非譲渡性	非リンク性	主な処理
内在型 [9], [10]	有り	可	BBS 検証
検証過程 ZK 型 [8]	有り	可	検証回路の ZKP
デバイス統合型 [11]	有り	可	デバイス内秘密値を用いた証明
JWT (4.1 節)	有り †	不可	Web2 署名検証
SD-JWT (4.2 節)	有り	不可	Web2 署名検証
BBS, KB あり (4.3 節)	有り	不可	BBS 検証
BBS, KB なし (4.3 節)	無し	可	BBS 検証

† 発行時に保有者鍵を束縛しないため弱い非譲渡性のみ持つ

3. システムモデル

本節では，本研究で想定するシステムを定義する．図 2 に，JWT，SD-JWT，BBS の各方式に共通する基本的なシーケンスを抽象化して示し，共通の処理はアルゴリズム 1 に示す通りである．

3.1 本研究のモデル

本研究では，ユーザ（Holder），発行者，検証者，および外部のチェーン RPC から構成されるシステムを想定する．本研究では発行者を信頼するものとする．本方式は，ユーザと発行者のみが関与する発行フェーズ（図 2 の (1)–(4)）と，ユーザと検証者のみが関与する提示フェーズ（同 (5)–(6)）から構成される．

ユーザ

ユーザは Web3 鍵ペア (pk_{web3}, sk_{web3}) を保持し，ウォレットアドレス `address` に対応する秘密鍵によって `wallet_challenge` への署名 `sig_wallet` を生成できる．ここで署名対象には，`wallet_challenge` と Web2 公開鍵証明書のハッシュ値 $h = \texttt{x5t\#S256}(\texttt{cert_web2})$ を連結した $\texttt{wallet_challenge}||h$ を用いる．さらにユーザは，同一チャレンジとアドレスを連結した $\texttt{wallet_challenge}||\texttt{address}$ に対して Web2 秘密鍵で署名し，`sig_web2` を生成する．これらにより，ユーザはウォレットの所持と Web2 鍵の所持とを同一のチャレンジのもとで発行者へ示す．また，ユーザは Web2 鍵ペア (pk_{web2}, sk_{web2}) と，必要に応じてその公開鍵証明書 `cert_web2` を保持し，これを発行者へ送付する．Web2 鍵は提示時の holder binding に利用され，`KBpayload` に対する署名から `KBproof` を生成する．提示時には，検証者の開示要求 $\mathcal{D}$ に基づき，発行済みクレデンシャルから提示データ `presentation` を生成する．

発行者

発行者は，あらかじめ述語集合[*2] $\mathcal{P} = (P_1, \ldots, P_n)$ を公開したうえで，ユーザのウォレット所持証明を確認し，チェーン RPC からウォレットに対応する `account_info` を取得して属性評価を行う．発行者はまず，ユーザから受

[*2] 本評価では，残高および取引回数に関する閾値判定からなる述語集合を用いる（例：balance_ge_0_1eth，tx_count_ge_10）．

© 2026 Information Processing Society of Japan

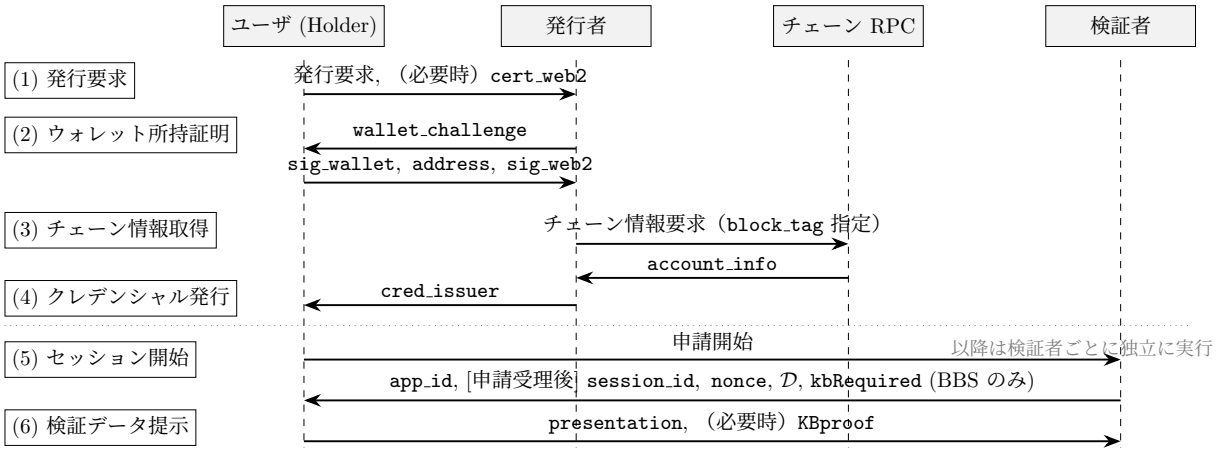


図 2 Web2/3 連携シーケンスの概要．ただし，(1) の cert_web2 送付は SD-JWT では必須，BBS では kbRequired = true のみ実行し，JWT では提示時に KBproof に含める．

アルゴリズム 1 Web2/3 連携 共通シーケンス

(0) 事前設定

Issuer: publish $\mathcal{P} = (P_1, \ldots, P_n)$

(1) 発行要求

User → Issuer: issuance request *and* cert_web2 *if required.*

(2) ウォレット所持証明

Issuer: wallet_challenge ← GenChallenge()
Issuer → User: wallet_challenge
User: $h \leftarrow$ x5t#S256(cert_web2)
User: sig_wallet ← SignWeb3(sk_{web3}, wallet_challenge||h)
User: sig_web2 ← SignWeb2(sk_{web2}, wallet_challenge||address)
User → Issuer: (sig_wallet, address, sig_web2)
Issuer: VerifyWeb3Sig(address, wallet_challenge||h, sig_wallet)
Issuer: VerifyWeb2Sig(pk_{web2}, wallet_challenge||address, sig_web2)

(3) チェーン情報取得

Issuer: block_tag ← finalized
Issuer → Chain RPC: request account information for address at block_tag
Chain RPC → Issuer: account_info

(4) クレデンシャル発行

Issuer:
  flags ← EvalPolicy($\mathcal{P}$, account_info) $\in \{0,1\}^n$
  payload ← Payload(flags, block_tag, address, cnf)
  where cnf = {x5t#S256 : h} *if required.*
  // iss, iat, exp, block_tag, cnf *are always disclosed;*
  // address *is withheld by default.*
  cred_issuer ← SignIssuer(sk_{issuer}, payload)
Issuer → User: cred_issuer

(5) セッション開始

User → Verifier: start session
Verifier: app_id ← GenAppID() *// random value.*
Verifier → User: app_id
Verifier: if the request is "accepted" then
  (session_id, nonce) ← GenSession() *// random values.*
  determine $\mathcal{D} \subseteq \{1, \ldots, n\}$ *// indices of the predicates.*
Verifier → User: (session_id, nonce, $\mathcal{D}$)
  and optionally kbRequired *for BBS.*

(6) 検証データ提示

User:
  presentation ← Presentation(cred_issuer, $\mathcal{D}$)
  sd_hash ← ProofDigest(presentation)
  // compute a SHA-256 digest over the presentation.
  KBpayload := (iat, aud, nonce, sd_hash)
  with the header typ = "kb+jwt".
  KBproof ← (SignWeb2(sk_{web2}, KBpayload), cert_web2) if required
User → Verifier: (presentation, KBproof)

(7) 検証

Verifier:
  VerifyPresentation(pk_{issuer}, presentation)
  sd_hash ← ProofDigest(presentation)
  // recompute the expected SHA-256 digest
  KBpayload := (iat, aud, nonce, sd_hash)
  VerifyKBproof(pk_{web2}, KBpayload, KBproof) if kbRequired
  // whether to require KB is decided in (5).
  check cnf matches x5t#S256(cert_web2) if present
  check now < exp and now − time(block_tag) < Δ
  evaluate its own policy on flags$|_{\mathcal{D}}$

け取る (sig_wallet, address, sig_web2) を用いて Web3 署名と Web2 署名を検証し，対象ウォレットの所持および Web2 鍵との対応を確認する．続いて，確定済みブロック block_tag を指定して取得した account_info に対して EvalPolicy($\mathcal{P}$, account_info) を適用し，各述語の判定結果 flags $\in \{0,1\}^n$ を求める．その後， payload に対して発行者秘密鍵 sk_{issuer} で署名し，クレデンシャル cred_issuer を生成する．ここで address は選択的開示の対象とし，通常の運用では検証者へ開示しない．また，holder binding をクレデンシャルに含める方式では，必要に応じて h = x5t#S256(cert_web2) を cnf として発行内容へ束縛する．なお，発行者は個々の検証者の判定条件を

© 2026 Information Processing Society of Japan

知る必要はなく，$\mathcal{P}$ に対する評価結果のみを発行する．

検証者

検証者は，ユーザによる提示を受け付け，クレデンシャルの妥当性および必要に応じた holder binding を検証する．検証者はまず申請を受理したユーザに対して `app_id` を返し，受理後に `session_id` と `nonce` を生成する．さらに，自身の判定に必要な述語の添字集合 $\mathcal{D} \subseteq \{1, \ldots, n\}$ を決定し，ユーザへ通知する．ここで `nonce` は提示情報の事前生成を防ぐため必ず検証者が生成し，`session_id` は検証者内部の状態管理にのみ用いる．

提示時には，検証者はユーザから `presentation` と，必要に応じて `KBproof` を受け取る．ここで `sd_hash` は ProofDigest(`presentation`) により導出され，これを含む `KBpayload` $=$ (`iat`, `aud`, `nonce`, `sd_hash`) が holder binding の検証対象となる．検証者は，まず発行者公開鍵 pk_{issuer} を用いて `presentation` の正当性を検証し，その後，`kbRequired` の場合には pk_{web2} を用いて `KBpayload` に対する Web2 署名を検証するとともに，`cnf` が `x5t#S256`(`cert_web2`) と一致することを確認する．最後に，`exp` および `block_tag` が示す時点からの経過時間を確認したうえで，$\texttt{flags}|_{\mathcal{D}}$ に対して自身のポリシーを評価する．これにより，提示内容の真正性と，必要に応じた提示主体との結び付け，および参照情報の鮮度を確認する．

3.2 安全性要件

本研究で要求される安全性を定性的に定義する．

S-1： 発行者署名の偽造不可性

正当な発行者以外が有効な認証情報を生成できないこと．発行者が生成するクレデンシャルは署名によって保護され，検証者はその正当性を検証できなければならない．

S-2： 開示属性の完全性

ユーザが提示する開示属性値は，発行時の属性値と一致していなければならない．開示内容が改ざんされた場合，検証者はそれを検出できる必要がある．

S-3： 提示主体の正当性

検証者の想定する提示主体がクレデンシャルを提示していると検証者が確認できること．

S-4： 非開示属性の秘匿性

ユーザが非開示を選択した属性について，提示情報からその値を推測できないこと．

S-5： 複数提示の非リンク性

同一クレデンシャルの複数回提示が，検証者により容易にリンクされないこと．

S-6： 複数検証者間の非リンク性

異なる検証者に対する提示が，検証者間で容易にリンクされないこと．

S-7： 検証コンテキストへの束縛

提示が特定の検証セッションにのみ有効であり，他の検証コンテキストへ容易に転用できないこと．

S-8： リプレイ耐性

過去に生成された提示情報を再利用して検証者を欺けないこと．本研究では，鍵の所持証明時には発行者が生成する wallet_challenge を署名対象に含め，提示時には検証者が生成する nonce を提示計算に含めることで，再利用を防止する．

S-9： Web2–Web3 間の鍵対応

発行対象の Web3 ウォレットに対応する鍵と，提示時に用いられる Web2 鍵との対応関係を確認できること．

S-10： 発行時参照情報の整合性

発行者が参照するウォレット関連情報が，一定の一貫性をもって取得されること．

3.3 脅威モデル

本研究では，3.1 節のとおり，発行者を信頼し，ユーザや検証者は完全には信頼できない攻撃者と想定する．攻撃者は通信の盗聴，改ざん，再送を行えるものとする．

T-1： 署名偽造

攻撃者が発行者署名または KB proof を偽造し，不正な認証情報を生成する攻撃である．この脅威は，発行者署名の偽造については S-1, KB proof の偽造については S-3 により対処する．

T-2： 開示情報改ざん

ユーザが開示する属性値を改ざんし，検証者を欺く攻撃である．これは開示属性の完全性（S-2）により防止される．

T-3： 不正な提示主体による利用

攻撃者が正当ユーザのクレデンシャルを入手し，本人になりすまして提示する攻撃である．この脅威に対しては提示主体の正当性（S-3）により対処する．

T-4： チャレンジ盗用

攻撃者が正当ユーザのチャレンジ値を再利用し，提示を偽装する攻撃である．この脅威はリプレイ耐性（S-8）によって防止される．

T-5： 別セッション・別検証者への再利用

過去に生成された提示情報を別セッションまたは別検証者に対して再利用する攻撃である．この脅威は検証コンテキストへの束縛（S-7）およびリプレイ耐性（S-8）によって抑制される．

T-6： 提示情報のリンク

ユーザの複数回提示を検証者が相関付ける攻撃である．この脅威への耐性は複数提示の非リンク性（S-5）および複数検証者間の非リンク性（S-6）に依存し，特に holder binding の運用方法が影響する．

T-7： Web2–Web3 鍵の組合せ不正

発行時に確認された Web3 ウォレットに対応する鍵と，提示時に用いられる Web2 鍵を不正に組み合わせる攻撃である．この脅威は提示主体の正当性（S-3）および

© 2026 Information Processing Society of Japan

Web2–Web3 間の鍵対応（S-9）によって抑制される．

T-8：参照情報の不正または不整合

悪意ある RPC 応答や不整合な参照情報により，誤った属性評価を発行者へ行わせる攻撃である．RPC の信頼性は本研究の比較対象外とし，発行時参照情報の整合性（S-10）をみたす信頼できる情報源が利用可能であると仮定する．

本研究で考慮する安全性

本研究では，利用する署名方式（JWT，SD-JWT，BBS 署名， Web2 鍵による署名）が標準的な偽造不可能性を満たすものと仮定する．ただし，上記要件のすべてを各方式が同時に満たすものではなく，4 節で方式ごとの差異を述べる．また，発行者が参照するウォレット関連情報は，チェーン RPC から取得される `account_info` が評価に利用可能な一貫した情報であるものとする．また，正当なウォレット所持者が自らの意思で第三者へ署名を提供する場合，発行者は両鍵が異なる主体に帰属することを検知できないため，本研究の対象外とする．

4. 提案シーケンス

本研究で提案する認証方式について，図 2 およびアルゴリズム 1 に示す共通フロー，ベースラインとして JWT，提案方式として SD-JWT，BBS 署名を用いた処理シーケンスを説明する．以下では，`account_info`, `cred_issuer`, `presentation`, `KBproof` に着目して各項目の差分を述べる．また，`KBproof` に含まれる `sd_hash` は方式ごとの正規化関数により計算される．

4.1 ベースライン：JWT ベース方式

JWT ベース方式では，図 2 を前提とするが (1) の Web2 証明書送付は行わない．そのため，(2) では $h \leftarrow$ `x5t#S256`(`cert_web2`) を用いた Web2 鍵との対応を行わず， `sig_web2` の生成・検証も行わないため，Web2 鍵の所持を確認しない．一方，提示時に `KBproof` とともに `cert_web2` を送付することで holder binding を実現する．ただし，発行時にクレデンシャルと Web2 鍵を対応付けていないため，非譲渡性は限定的である．なお，(3)，(5) は共通シーケンスと同様に実行される．

(3) において発行者が取得する `account_info` は，主として参照ブロックとウォレット残高情報であり，

$$\texttt{account_info} = (\texttt{block_tag}, \texttt{balance_wei}, \texttt{balance_eth})$$

として扱う．したがって，(4) で発行される `cred_issuer` は発行者署名付き JWT であり，

$$\texttt{cred_issuer} = \texttt{JWT}(\texttt{iss}, \texttt{iat}, \texttt{exp}, \texttt{chain_id}, \texttt{block_tag}, \texttt{address}, \texttt{flags})$$

に相当する．

(6) において，ユーザは

$$\texttt{presentation} = \texttt{cred_issuer}$$

として JWT 全体を提示する．また，

$$\texttt{KBproof} = (\texttt{payload}, \texttt{sig_web2}, \texttt{cert_web2})$$

であり，

$$\texttt{payload} = (\texttt{iat}, \texttt{aud}, \texttt{nonce}, \texttt{sd_hash}),$$
$$\texttt{sd_hash} = H(\texttt{JWT})$$

とする．検証者は，JWT の発行者署名，`nonce` の整合性，`KBproof` に含まれる Web2 署名，および `exp` と `block_tag` が示す参照時点の鮮度を検証する．

この方式は既存 JWT 基盤との親和性が高く構成も単純であるが，JWT 全体を送信するため選択的開示は実現できない．このため検証者は，自身が要求した $\mathcal{D}$ 以外の述語判定結果および `address` も受け取ることとなり，1 節で述べたウォレットアドレス非開示の要件を満たさない．

4.2 SD-JWT ベース方式

SD-JWT ベース方式では，発行者署名付き JWT に disclosures を組合せて必要な属性のみを検証者へ提示する．

本方式では，図 2 の (2)，(3)，(5) は共通シーケンスと同様に実行される．また，(1) においてユーザはあらかじめ Web2 証明書を発行者へ送付し，そのハッシュ値を発行時の holder binding に利用する．

(3) において発行者が取得する `account_info` は，`eth_getProof` の結果を要約した情報であり，

$$\texttt{account_info} = (\texttt{block_tag}, \texttt{balance_wei}, \texttt{account_proof_digest})$$

として扱う．(4) で発行される `cred_issuer` は

$$\texttt{cred_issuer} = \texttt{SD-JWT}(\texttt{issuerJwt}, \texttt{disclosures})$$

に相当し，`issuerJwt` には常時開示クレームとして `iss`，`iat`，`exp`，`block_tag`，および `cnf.x5t#S256` が含まれ，各述語の判定結果 `flags` と `address` が選択的開示の対象となる．

(6) において，ユーザは検証者から通知された $\mathcal{D}$ に対応する disclosures のみを選択し，

$$\texttt{presentation} = (\texttt{issuerJwt}, \texttt{selectedDisclosures})$$

として送信する．さらに，

$$\texttt{KBproof} = (\texttt{payload}, \texttt{sig_web2}, \texttt{cert_web2})$$

であり，

$$\texttt{payload} = (\texttt{iat}, \texttt{aud}, \texttt{nonce}, \texttt{sd_hash}),$$
$$\texttt{sd_hash} = H(\texttt{presentation})$$

とする[*3]．検証者は，発行者署名付き SD-JWT の正当性，選択された disclosures の整合性，`KBproof` の妥当性，証明書ハッシュ値と `cnf.x5t#S256` の一致，および参照時点の鮮度を検証する．

この方式により，JWT 系フォーマットを維持したまま選択的開示を実現できる．また `KBproof` は RFC 9901 が規定する key binding JWT と同一の構造を持つため，既

[*3] `sd_hash` は RFC 9901 の 4.3 節に従い，`issuerJwt` と選択された disclosures をチルダで連結した文字列に対して計算する．

© 2026 Information Processing Society of Japan

存の SD-JWT 実装をそのまま利用できる．

4.3 BBS ベース方式

BBS ベース方式では，発行者が BBS 署名付きクレデンシャルを発行し，ユーザはそこから選択的開示 proof を生成して検証者に提示する．本実装では可搬性のため `header.payload.signature` の JWT ライクなシリアライズを採用するが，検証は BBS の `ProofVerify` を用いる．

本方式では，図 2 の (2)，(3)，(5) は共通シーケンスと同様に実行される．また，(1) の Web2 証明書送付は `kbRequired=true` の場合にのみ実行される．

(3) において発行者が取得する `account_info` は SD-JWT と同様に

$$\texttt{account_info} = (\texttt{block_tag}, \texttt{balance_wei}, \texttt{account_proof_digest})$$

とする．したがって (4) で発行される `cred_issuer` は

$$\texttt{cred_issuer} = (\texttt{header}, \texttt{payload}, \texttt{signature}_{\mathrm{BBS}})$$

に相当する．ここで `payload` は `message_order` と `claims` を含み，

$$\texttt{claims} = (\texttt{address}, \texttt{chain_id}, \texttt{block_tag}, \texttt{flags})$$

として表現される．`message_order` は述語集合 $\mathcal{P}$ に対応する固定順とし，全ユーザで共通とする．また，`kbRequired=true` の場合には Holder の Web2 証明書ハッシュ値を holder binding 用情報として含める．

(6) でユーザは $\mathcal{D}$ に対応する開示クレームのみを選択し，元の BBS 署名から選択的開示 proof を生成するため，

$$\texttt{presentation} = (\texttt{disclosed_indexes}, \texttt{disclosed_claims}, \texttt{proof_b64url})$$

である．検証者は `proof_b64url` に対して BBS の `ProofVerify` を実行することで，非開示属性を受け取ることなく開示属性のみを検証できる．

さらに，`kbRequired=true` の場合には，

$$\texttt{KBproof} = (\texttt{payload}, \texttt{sig_web2}, \texttt{cert_web2})$$

を追加送信する．ここで

$$\texttt{payload} = (\texttt{iat}, \texttt{aud}, \texttt{nonce}, \texttt{sd_hash}),$$

$$\texttt{sd_hash} = H(\texttt{presentation})$$

である．一方，`kbRequired=false` の場合には `KBproof` は送信されず，BBS proof のみで検証を行う．

この方式は選択的開示に加えて，検証者の要求に応じて holder binding を付与できる．特に `kbRequired=false` では複数提示間の非リンク性を維持しやすく，`true` ではその代わりに holder binding を強化できる．いずれの場合も BBS の presentation header には (`aud`, `nonce`) を与え，提示を検証セッションへ束縛する．

5. 概念実証による評価

本節では，JWT，SD-JWT，BBS ベース方式について，同一の認証シナリオのもとで概念実証を行い，処理時間および提示データサイズの観点から比較評価する．

5.1 設定

概念実証は TypeScript/Node.js 上に実装し，発行者および検証者は HTTPS サーバとして動作させた．BBS 署名処理には `microsoft/bbs-node-reference`[*4]を利用した．計測は同一 API フローを自動実行するベンチマークスクリプトにより行い，チェーン情報取得には外部 RPC 通信の揺らぎを除外するため mock chain モードを用いた．計測環境は，macOS 26.5.2，Apple M3 Max（16 論理コア），64 GB メモリ，Node.js v22.17.1，TypeScript 5.9.3，`ethers.js` 6.16.0，`@sd-jwt/core` 0.19.0 である．

各方式は同一の認証シナリオのもとで評価し，発行者は 3 節で述べた $n = 8$ の述語を評価してクレデンシャルを発行する．検証者の開示要求は $\mathcal{D} = \{1\}$（`balance_ge_0_1eth`）とし，ウォレット所持証明は全方式で発行前に実施した．各計測に先立ち 100 回のウォームアップを実行し，試行回数は JWT および SD-JWT で n = 1000，BBS で n = 100 とした．

暗号パラメータとして，JWT および SD-JWT の発行者署名，KB-JWT の署名には ES256 を用い，発行者の鍵および Web2 鍵はともに NIST P-256（prime256v1）上の ECDSA 鍵とした．Web3 側のウォレット所持証明には Ethereum の `personal_sign` を用い，secp256k1 上の ECDSA 署名として検証した．`x5t#S256` は X.509 証明書 DER に対する SHA-256 ハッシュ値として算出した．BBS 署名には BLS12-381 系 ciphersuite を用い，本評価では既定値 `BLS12_381_SHAKE_256` を利用した．なお，HTTPS 通信路には自己署名 X.509 証明書を用いたが，TLS 証明書方式自体は比較対象に含めていない．

評価項目は，発行時間，提示生成時間，検証時間，発行クレデンシャルサイズ，提示データサイズ，送信データ全体サイズである．ここで提示データサイズは，`presentation` 自体の UTF-8 バイト長とした．BBS 方式では，常時開示クレームを保持する `header` バイト列も提示データに含めている．送信データ全体サイズは，これに `KBproof` および証明書を加えた送信 JSON 全体の UTF-8 バイト長とした．また，検証時間については，方式固有の検証処理を表す「クレデンシャル検証」と，提示主体との束縛確認を表す「KB 検証」に分けて記録した．

5.2 結果

表 2 に各方式の処理時間およびデータサイズを示す．「検証全体」はクレデンシャル検証および KB 検証以外に，入力処理，ダイジェスト再計算，鮮度判定，判定結果生成等の周辺処理を含む全体の処理時間のため，両者の合計値と

[*4] https://github.com/microsoft/bbs-node-reference （CFRG draft-irtf-cfrg-bbs-signatures-09 相当）

© 2026 Information Processing Society of Japan

表 2 各方式の処理時間およびデータサイズ

	時間 [ms]					データサイズ [byte]		
方式	発行	提示生成	検証全体	クレデンシャル検証	KB 検証	発行クレデンシャル	提示データ	送信データ全体
JWT	2.43	0.89	0.83	0.10	0.26	532	532	1496
SD-JWT	2.58	0.52	0.95	0.21	0.26	1451	1011	1963
BBS (KB あり)	57.67	119.19	99.20	97.65	0.40	874	1170	2132
BBS (KB なし)	57.64	118.36	98.56	97.36	—	787	1083	1132

は一致しないことに注意されたい．

JWT および SD-JWT は，発行・提示生成・検証のいずれも数 ms 以下で完了し，軽量な処理であることが確認できた．クレデンシャル検証および KB 検証は JWT，SD-JWT いずれも 1 ms 未満であり，両方式の検証コストは同程度に小さい．一方，BBS は他方式に比べて発行時間，提示生成時間，検証時間のいずれも大きく，特に提示生成および検証では約 100 ms を要した．また，検証時間の大部分は BBS proof の検証に起因していることが分かる．

サイズの観点では，JWT は発行クレデンシャルおよび提示データが最も小さい．SD-JWT は disclosures を含むため発行クレデンシャルサイズが 1451 byte と最も大きい一方，選択的開示により提示データサイズは 1011 byte に減少する．BBS は提示データサイズが 1170 byte（`kbRequired=false` では 1083 byte）であり，SD-JWT を上回った．これは BBS proof に加えて，常時開示クレームを保持する `header` バイト列を提示データへ転記する必要があるためである．

以上より，JWT は低コストだが全開示であり，SD-JWT は既存 JWT 基盤と整合的な選択的開示を低コストで実現する方式，BBS は約 2 桁大きい計算コストと引き換えに，発行クレデンシャルサイズの削減と非リンク性制御の柔軟性を提供する方式と整理できる．ただし，提示データサイズでは BBS は常に SD-JWT を上回り，送信データ全体で BBS が SD-JWT を下回るのは holder binding を放棄した場合に限られる．

6. おわりに

本研究では，Web3 情報と Web2 PKI 認証基盤を連携させ，ユーザのウォレット情報に基づく属性を安全に提示・検証するための三者モデルを提案した．提案方式は発行フェーズと提示フェーズを分離し，単一のクレデンシャルを複数の検証者へ提示可能とする．さらに，認証情報の実現方式として，JWT，SD-JWT，BBS 署名の三方式を対象とし，それぞれの認証シーケンスを設計するとともに，安全性要件および脅威モデルとの対応関係を整理した．実装評価より，JWT 方式は実装が容易で処理性能に優れる一方，選択的開示や非リンク性といったプライバシ保護機能を提供できないこと，SD-JWT 方式は選択的開示を実現し既存の Web 技術との親和性を維持できること，BBS 方式は発行クレデンシャルを小さく保てるものの，約 2 桁大きい計算コストを要し，複数提示間の非リンク性は holder binding を課さない場合に得られることを明らかにした．

今後の課題として，各安全性要件を形式的に定義し，攻撃モデルに基づく安全性検証を通じて，提案方式の安全性を体系的に評価することが挙げられる．また，実サービスへの適用を見据え，大規模環境における性能評価や運用上の責任分担の設計についても検討を進める予定である．

参考文献

[1] Financial Action Task Force. Updated guidance for a risk-based approach to virtual assets and virtual asset service providers. Technical report, FATF, 2021.

[2] 金融庁. マネー・ローンダリング及びテロ資金供与対策に関するガイドライン. Technical report, 金融庁, 2026.

[3] Sarah Meiklejohn, Marjori Pomarole, Grant Jordan, Kirill Levchenko, Damon McCoy, Geoffrey M. Voelker, and Stefan Savage. A fistful of bitcoins: characterizing payments among men with no names. *Commun. ACM*, 59(4):86–93, 2016.

[4] Manu Sporny, Ted Thibodeau Jr, Ivan Herman, Gabe Cohen, and Michael B. Jones. Verifiable Credentials Data Model v2.0. W3C Recommendation, May 2025.

[5] Daniel Fett, Kristina Yasuda, and Brian Campbell. Selective Disclosure for JSON Web Tokens. RFC 9901, November 2025. IETF Standards Track.

[6] Tobias Looker, Vasilis Kalos, Andrew Whitehead, and Mike Lodder. The BBS Signature Scheme. Internet-Draft, IRTF CFRG, January 2026. draft-irtf-cfrg-bbs-signatures-10.

[7] Greg Bernstein and Manu Sporny. Data Integrity BBS Cryptosuites v1.0. `https://www.w3.org/TR/vc-di-bbs/`, April 2026.

[8] Christian Paquin, Guru-Vamsi Policharla, and Greg Zaverucha. Crescent: Stronger Privacy for Existing Credentials. IACR Cryptology ePrint Archive, Report 2024/2013, 2024.

[9] Anja Lehmann, Andrey Sidorenko, and Alexandros Zacharakis. Vision: A Modular Framework for Anonymous Credential Systems. IACR Cryptology ePrint Archive, Report 2025/1981, 2025.

[10] Dan Yamamoto, Yuji Suga, and Kazue Sako. Formalising linked-data based verifiable credentials for selective disclosure. In *EuroS&P 2022 - Workshops*, pages 52–65. IEEE, 2022.

[11] Julia Hesse, Nitin Singh, and Alessandro Sorniotti. How to Bind Anonymous Credentials to Humans. In *USENIX Security 23*, pages 3047–3064. USENIX Association, 2023.

© 2026 Information Processing Society of Japan